INTRODUCTION

Credit risk assessment is a fundamental task in the financial services industry. When a bank issues a loan, it faces a risk that borrower will fail to meet repayment obligation (a loan default). Accurate prediction of default risk enables these institution to make better lending decision, setting the appropriate interest rates, and maintain portfolio health. Traditional credit scoring relies on static rule based systems, whereas modern data mining techniques offer better data driven risk assessment.

This project applies five classification algorithms to a dataset of 35,000 historical loan applications, each described by 20 attributes spanning demographic, financial, and temporal characteristics. The target variable, *loan_default*, is binary: **1** indicates the borrower defaulted, **0** indicates successful repayment. The dataset exhibits moderate class imbalance (61.3% non-default, 38.7% default), substantial missingness in several features (up to 16.4%), and extreme scale differences across numerical attributes. These challenges necessitate careful preprocessing, feature engineering, and model selection.

The project follows a structured data mining workflow: data understanding, preprocessing and transformation, exploratory data analysis, feature engineering, model development with hyperparameter tuning, evaluation and comparison, and finally prediction on an unlabelled dataset for Kaggle competition submission.

PROBLEM DEFINITION

The task is a supervised binary classification problem given a set of applicant features at the time of loan application, predict whether the loan will default (class 1) or be repaid successfully (class 0). The labelled dataset contains 35,000 observations with 20 attributes and a known target variable. A separate unknown dataset of 15,000 observations (identical features, no target) requires prediction for Kaggle evaluation.

From a business perspective, the cost of a false negative (approving a borrower who defaults) is typically higher than the cost of a false positive (rejecting a creditworthy applicant). This asymmetry informs the evaluation strategy: while accuracy provides a baseline, metrics such as F1-score, ROC-AUC, and recall for the default class are prioritised. The majority-class baseline accuracy is 61.3%, meaning any useful model must substantially exceed this threshold.

HOW THE PROBLEM WAS SOLVED

The work proceeded in five stages. First, both datasets were audited for shape, data types, missing values, and schema consistency. This is where the date format mismatch and the MCAR missingness pattern (class-differential gaps all under 2.7%) were identified. Second, missing values were imputed, categoricals encoded, and dates parsed. Exploratory analysis at this stage included Mann-Whitney U tests on numerical features and chi-squared tests on categoricals, establishing that credit_score and debt_to_income_pct were the strongest raw predictors while all categorical variables were insignificant. Third, five classifiers were trained on a stratified 80/20 split (28,000 for training, 7,000 for testing). Hyperparameters for

each model were tuned with 3-fold stratified cross-validation on the training partition. The test set was not used for any training or tuning decision.

Fourth, all five models were evaluated on the same 7,000-sample test set. McNemar's test checked whether the difference between the top two models was statistically significant. Fifth, each model was retrained on all 35,000 labelled rows and used to produce predictions on the 15,000 unknown samples for Kaggle submission.

Data Preprocessing

The labelled dataset comprises 35,000 rows and 20 columns. Ten features are numerical (age, income, credit_score, loan_amount, debt_to_income_pct, num_dependents, years_employed, account_balance, num_prev_loans, monthly_expenses), six are categorical (gender, employment_status, education_level, loan_purpose, region, marital_status), one is a date (application_date), and two are temporal integers (application_year, loan_duration_months). The target variable loan_default is binary.

Table 1 summarises the missingness profile observed in the labelled dataset.

Feature	Missing Count	Missing %	Type
education_level	5,728	16.4%	Categorical
marital_status	4,216	12.1%	Categorical
monthly_expenses	3,256	9.3%	Numerical
years_employed	2,683	7.7%	Numerical
account_balance	2,407	6.9%	Numerical
income	2,086	6.0%	Numerical
num_dependents	1,816	5.2%	Numerical

Table 1: Missingness profile of the labelled dataset.

A critical finding during data understanding was that missingness rates are nearly identical between the labelled and unknown datasets, and the rates do not differ significantly between default and non-default classes (all class-differential gaps < 2.7%). This pattern is consistent with Missing Completely at Random (MCAR), indicating that missingness is driven by systematic data-collection procedures (likely optional form fields) rather than being informative of default risk.

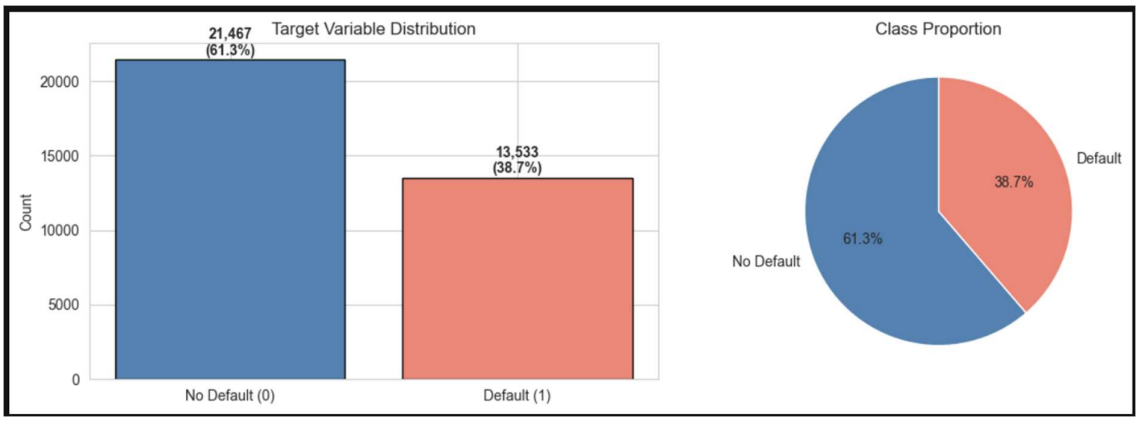
Missing Value Treatment

For numerical features (income, years_employed, account_balance, monthly_expenses, num_dependents), median imputation was applied. The median was chosen over the mean because several financial features exhibit extreme right skew (income skew = 2.28, account_balance skew = 8.25); the mean would be inflated by outliers and would not represent the central tendency accurately. Critically, median values were computed from the labelled training data only and then applied to both training and unknown datasets, preventing target leakage.

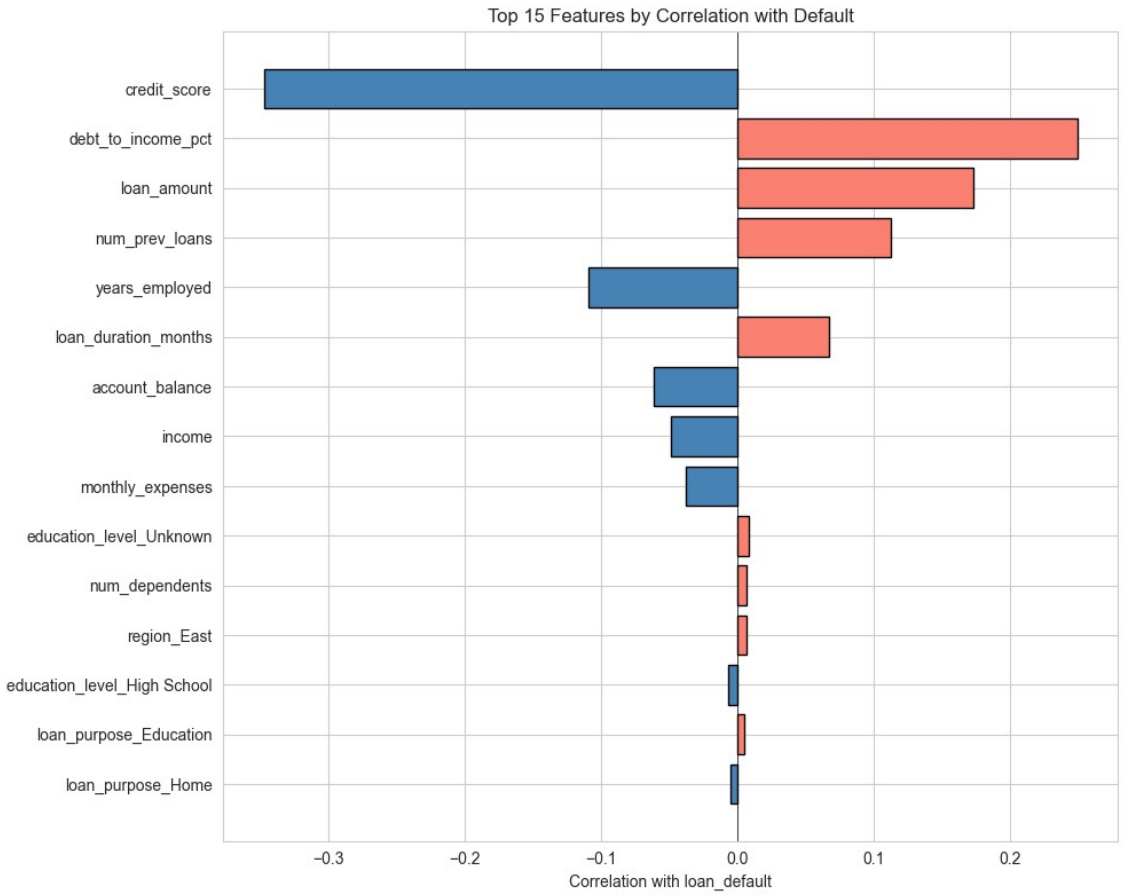
For categorical features (education_level at 16.4% missing, marital_status at 12.1% missing), an explicit 'Unknown' category was created rather than imputing the mode. With 16.4% of education_level values absent, mode imputation would fabricate 5,728 records with 'Bachelor' (the mode), diluting the true distribution and erasing the informational signal that the data was absent. The 'Unknown' category preserves this signal and allows the model to learn whether absence itself correlates with default.

Before imputation, binary missingness indicator features were created for all seven features with missing values. These indicators capture whether the original value was absent (1) or present (0). If applicants who leave fields blank tend to default more frequently, these indicators provide the model with direct access to that signal.

Exploratory Data Analysis



The target variable exhibits moderate class imbalance: 21,467 non-defaults (61.3%) versus 13,533 defaults (38.7%), yielding an imbalance ratio of 1.59:1. This is not extreme enough to warrant aggressive resampling (such as SMOTE), but it is sufficient to bias models that optimise raw accuracy. The class_weight='balanced' parameter was used in applicable models (Decision Tree, Random Forest, SVM) to address this.



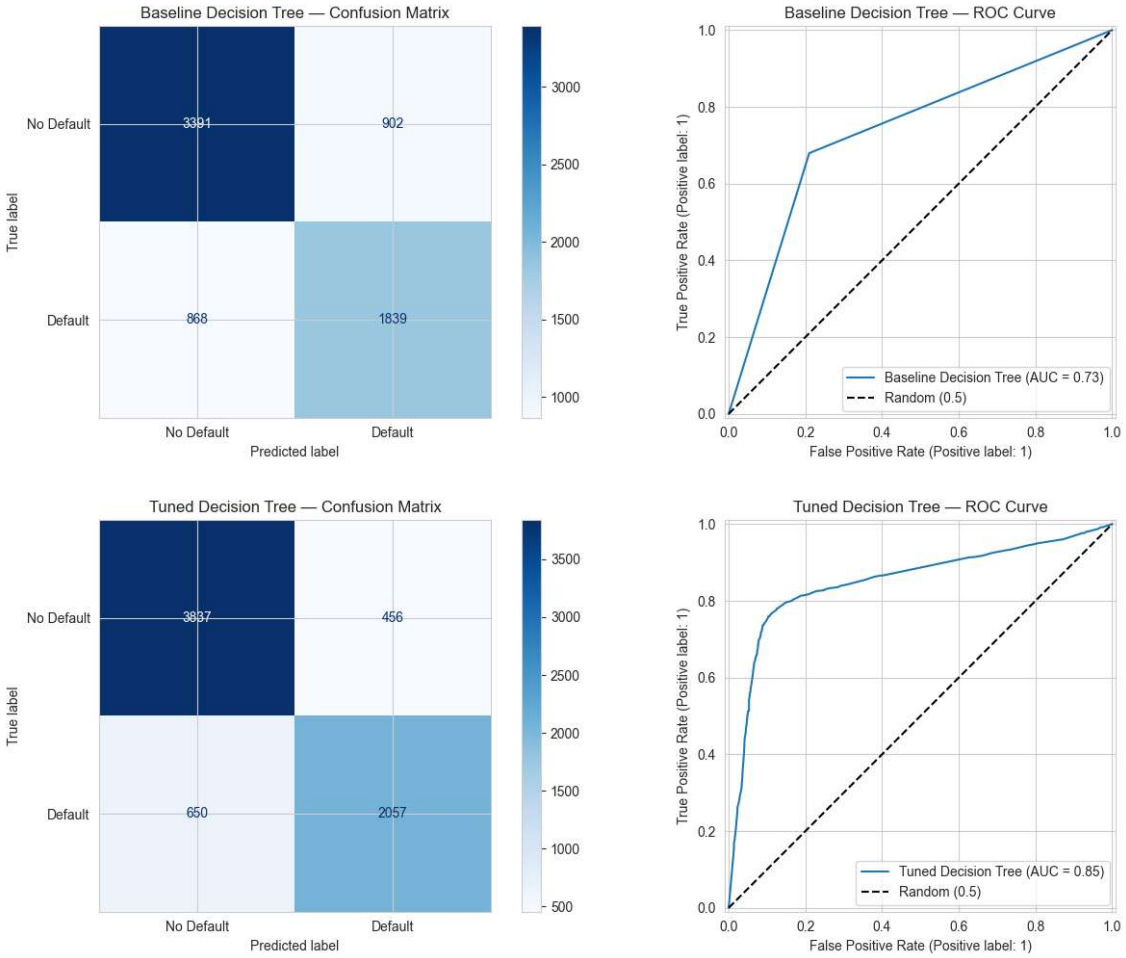
Correlation analysis identified credit_score as the single strongest predictor of default (Pearson r = −0.347), followed by debt_to_income_pct (r = +0.249) and loan_to_income (r = +0.195). Mann-Whitney U tests confirmed that most numerical features differ significantly between default and non-default classes ($p < 0.001$). Conversely, chi-squared tests revealed that all categorical features have negligible association with the target (all $p > 0.5$, all Cramér's $V < 0.01$). This was a pivotal finding: it indicated that the categorical features are essentially noise, and that the models would need to rely almost entirely on numerical and engineered features.

A severe multicollinearity was detected between income and monthly_expenses (r = 0.91), meaning one feature is largely redundant. The default rate was stable across application years (2010–2024) at approximately 37–41%, indicating no temporal trend that would complicate time-series effects.

METHODOLOGY

DECISION TREE

This model were trained with 80/20 split (28000 training / 7000 testing). First we set an unrestricted tree with no depth or leaf restriction applied. This allowed the tree to grow until every leaf only contained one sample, we deliberately induce maximum overfitting to quantify the gap between training and testing performance. A decision tree without restriction will partition the training data until every leaf is homogeneous, achieving close to 100% training accuracy. This behaviour then treats statistical noise as signal



Second stage is testing the tuned tree. Parameter grid explored (1) criterion: gini, entropy; (2) max depth: 5, 7, 9, 11, 13, 15; (3) min samples leaf: 1, 5, 10, 15, 25; (4) min samples split: 2, 10, 20. There are 180 combination tested and evaluated with 3 fold stratified cross validation and f1_macro is used as the scoring metric. From the testing the parameter with the best result are (1) criterion: entropy; (2) max depth: 9; (3) min samples leaf: 15; (4) min samples split: 20.

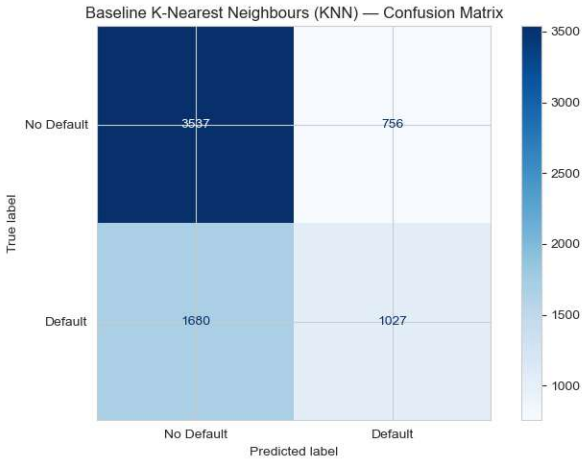
We can interpret the result of the trained tree as, The tuned tree correctly identifies 76.65% of defaults which mean the bank would catch around 76 out of every 100 risky

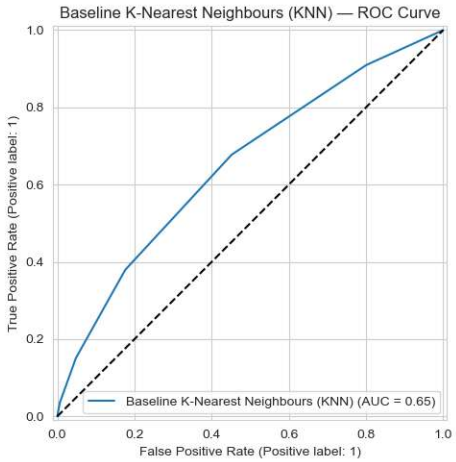
borrowers before giving loan. False alarm rate is about 10.6% of "approved looking customers (456/4293)", it is a significant business cost however still tolerable because the bank would avoid default losses.

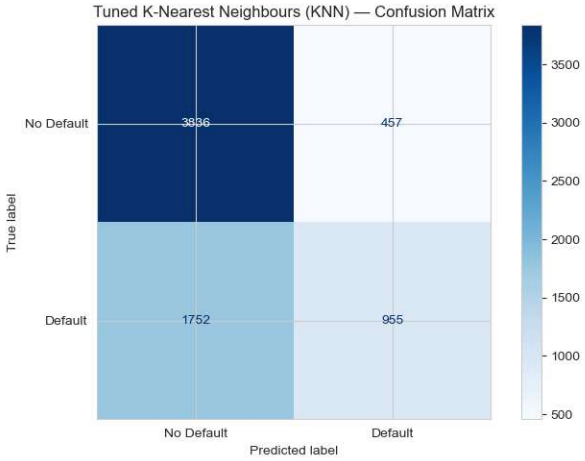
Metric	Base	Tuned tree	Improvement
Accuracy	0.7277	0.8164	+8.87%
Precision	0.6449	0.7606	+11.57%
Recall	0.6586	0.7665	+10.79%
F1-Score (default)	0.6517	0.7636	+11.19%
Fi-Score (Macro)	0.7150	0.8068	+9.18%
ROC-AUC	0.7150	0.8365	+12.15%
Tree Depth	39	9	-30 levels
Leaf Nodes	4285	304	-93%
Overfitting gaps	0.02723	0.0220	-92% reduction

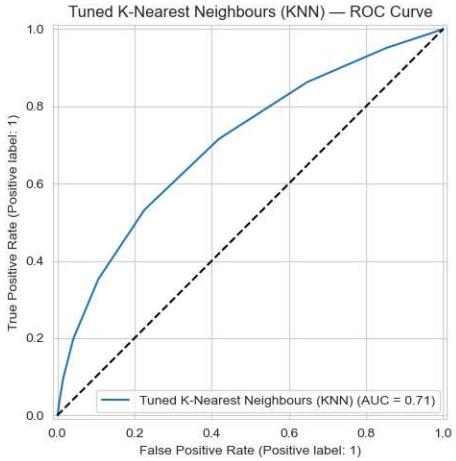
K-Nearest Neighbours (KNN)

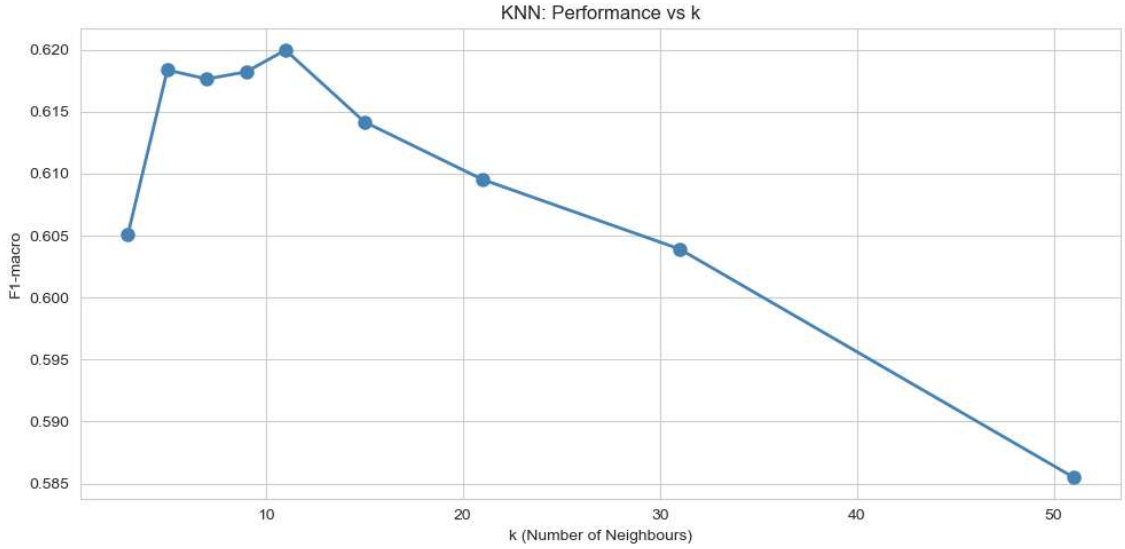
KNN classifier is set with scikit learns default settings which is n_neighbors=5, metric='eucledian', weights='uniform'. This is then trained on the 80% scaled training and scaling was applied because KNN is a distance based classifier with out it income will dominate attributes like num_dependents. The baseline is missing 62% of defaulter and an AUC score of 0.65 falls into the "poor to fair" band for credit scoring which is way below banking norm of 0.75+. Reading from ROC curves show that The curve bows modestly above the dashed diagonal. The diagonal represents random guessing (AUC=0.5). Not to mention the curve climbs steeply only in the very low FPR region then flattens which mean the model can only confidently identify a mall slice of clear defaulters before it become uncertain.











Second stage is testing the tuned tree. Parameter explored (1) n_neighbhors: 5, 11, 21, 31, 51; (2) metric: euclidean, manhattan; (3) weights: uniform, distance. From the testing, the parameter with the best result is (1) n_neighbhors: 11; (2) metric: manhattan; (3) weights: distance. ROC curve reading shows that the curved bows Is visibly higher than this baseline

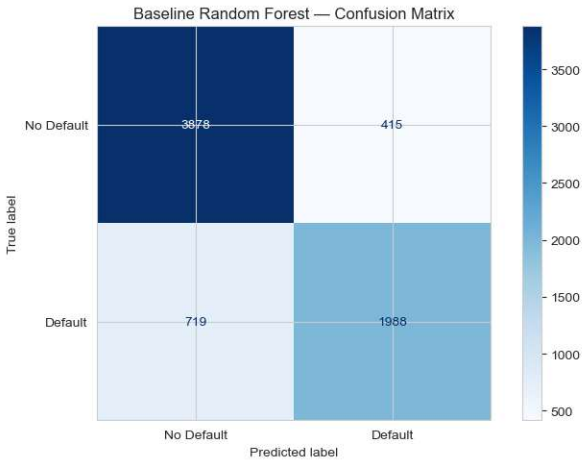
and it is above the baseline ROC at almost every threshold. AUC score is 0.71 a jump from baseline AUC which is 0.65. The early FPR region curves rises more steeply indicating the model is better at confidently identifying clear defaulters.

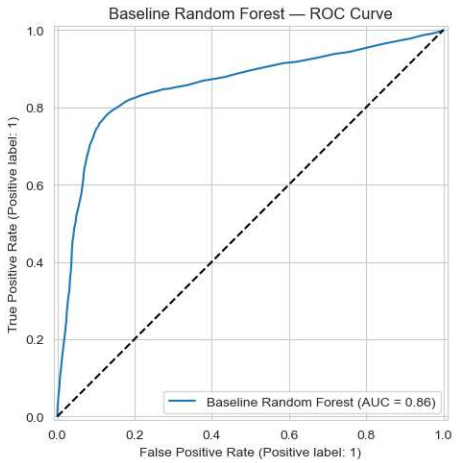
Metric	Base	Tuned tree	Improvement
Accuracy	65.2%	68.4%	+3.2%
Precision	57.6%	37.6%	+10%
Recall	37.9%	35.3%	-2.6%
F1-Score (default)	45.8%	46.4%	+0.6%
Specificity	82.4%	89.4%	+7%
ROC-AUC	0.65	0.71	+0.06

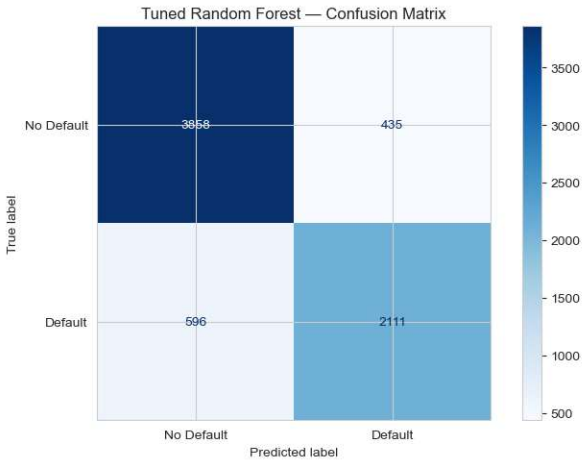
Of 2,707 actual defaulters in the test set, the tuned model catches about 955 (35.3%) and misses 1,752 (64.7%) defaulters. At a typical $20,000 average loan with a ~60% loss-given-default, those 1,752 missed defaulters represent on the order of **$21M in unmitigated risk** in this test data alone, and assuming the bank processes about 75,000 applications per year if we annualise from the 50k dataset. Of 4,293 actual non-defaulters, the model wrongly rejects 457 (10.6%). At an average customer lifetime value of, say, $5,000, that's about **$2.3M in lost revenue** from false alarms.

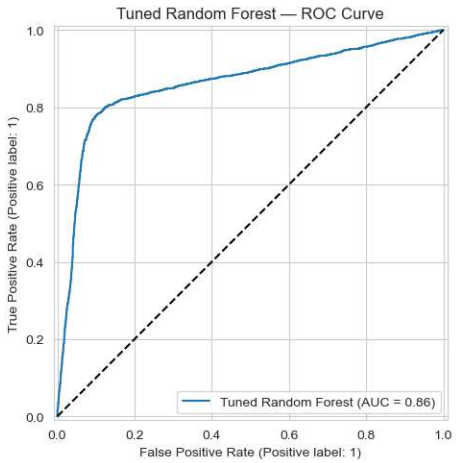
Random Forest (RF)

The first step is to create a baseline Random Forest to establish a reference point. Using sklearn default settings. Using GridSearchCV, we tries every combination of parameters using cross validation method to pick the best parameters. The best parameters that we got is as follows; (1) max depth: 15; (2) max features: 0.5; (3) min samples leaf: 5; (4) n estimators: 100.









Metric	Base	Tuned tree	Improvement
Accuracy	0.8304	0.8337	+0.33%
Precision	0.8133	0.7848	-2.85%
Recall	0.7289	0.7854	+5.65%
F1-Score (default)	0.7688	0.7851	+1.63%
F1-Score (macro)	0.8174	0.8247	+0.73%
ROC-AUC	0.8522	0.8544	+0.22%

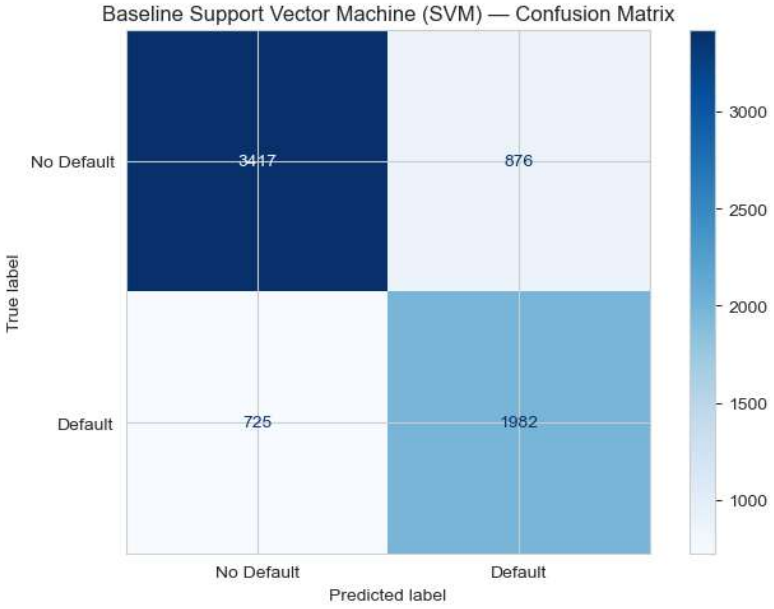
Observing the ROC curve and comparison table the baseline model and tuned model barely make any improvements. AUC went from 0.8522 to 0.8544 are essentially the same,

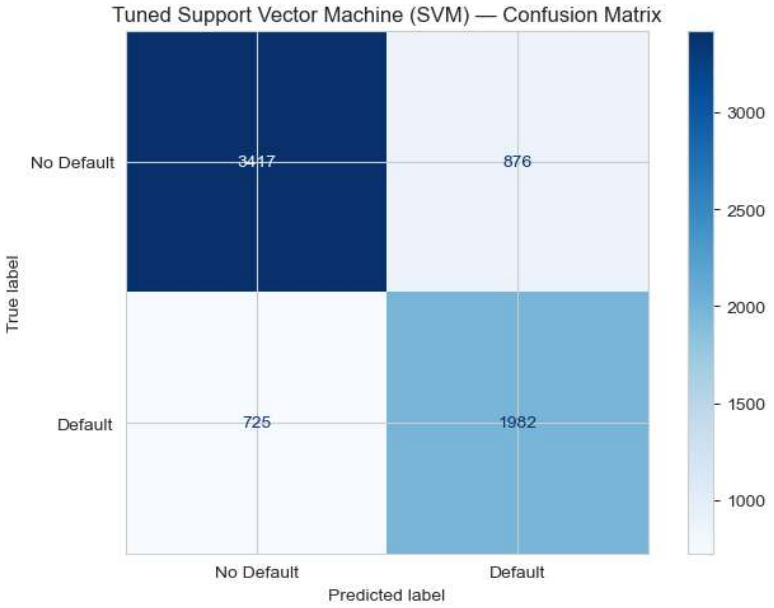
this means tunning the models did not make It better at ranking customers by risk. Both versions create roughly the same list from “most likely to default” to “less likely to default.” However, the tunning does make it better at setting the “cut off line”. The baseline only labels someone as default when it’s sure, while the tuned model is more willing to flag more people as risky which explain why it has lower precision score but higher recall score. The trade off of this is that, while we are losing 20 customers from false alarm, we are catching 123 more defaulters.

SVM

SVM looks for the boundary between the two classes that has the widest possible margin. With the RBF kernel, the algorithm maps the data into a higher-dimensional space where non-linear patterns become linearly separable.

Training SVM scales really poorly with dataset size (roughly proportional to n squared), so hyperparameter tuning was carried out on a random stratified subsample of 8,000 observations. The regularisation parameter C was swept from 0.1 to 10, revealing a peaked curve with C = 2.0 at the top. RBF beat both the linear kernel (AUC 0.826 vs 0.844). The final model, trained on all 28,000 rows (about 35 seconds), uses 14,231 support vectors, which is roughly half the training data. That high proportion suggests the two classes overlap considerably in feature space. SVM produces the most false positives (734 out of 7,000 test samples), pointing to a boundary that is too generous in labelling applicants as likely defaulters.



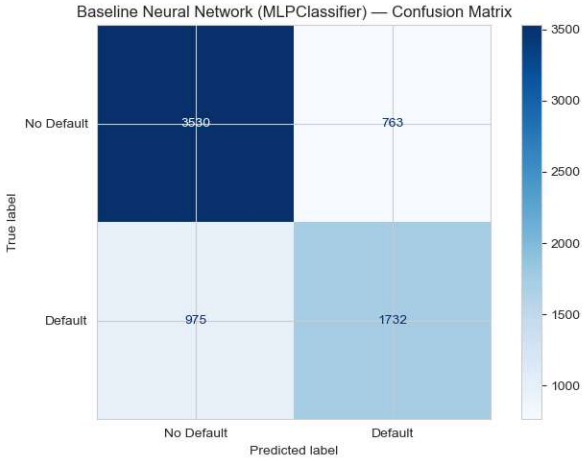


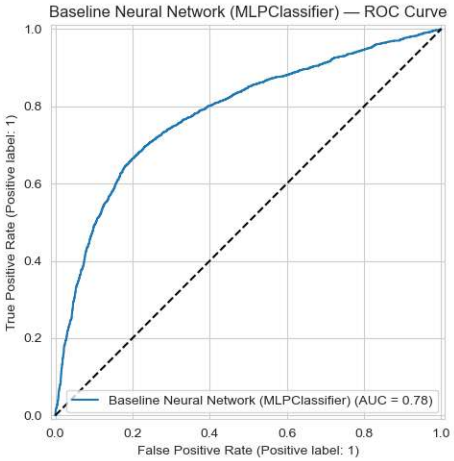
Neural Network

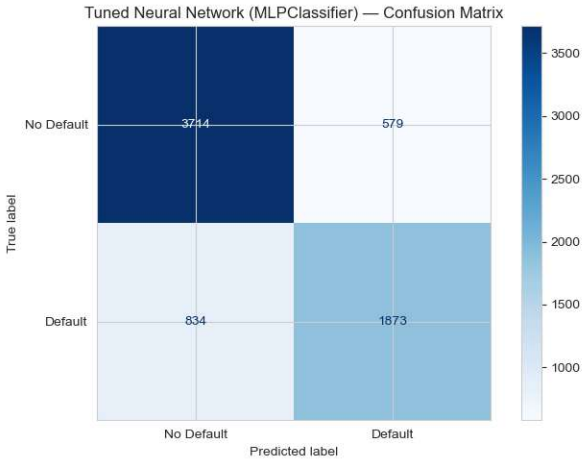
The NN classifier is set with scikit-learn's default MLP settings, specifically hidden_layer_sizes=(64, 32), activation='relu', solver='adam', alpha=0.0001, learning_rate_init=0.001, max_iter=200, no early stopping. This is then trained on the 80% scaled training set, and scaling was applied because the neural network computes weighted sums at every neuron without standardisation, income (range ≈ $20k–$500k) would numerically dominate features like num_dependents (range 0–5) regardless of their true predictive value, and gradient descent would spend the early epochs simply rescaling its way out of that imbalance.

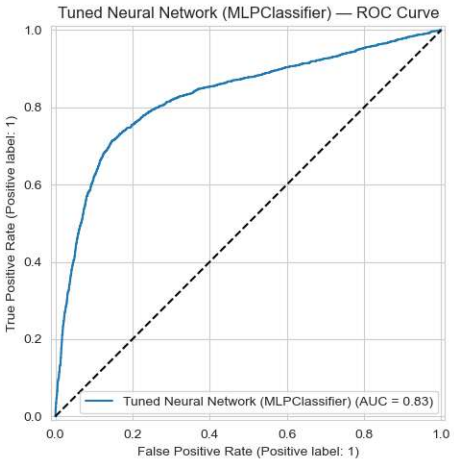
Second stage is testing the tuned network. Parameters explored: (1) hidden_layer_sizes: (64,), (64, 32), (128, 64); (2) activation: relu, tanh; (3) alpha: 0.0001, 0.001, 0.01; (4)

learning_rate_init: 0.001; with early_stopping=True (patience=15) and 3-fold stratified cross-validation scored on F1-macro. From the testing, the parameters with the best result are (1) hidden_layer_sizes: (128, 64); (2) activation: tanh; (3) alpha: 0.01; (4) learning_rate_init: 0.001. ROC curve reading shows that the tuned curve bows visibly higher than the baseline in the operating region (FPR 0.05–0.25) where credit decisions actually live and sits above the baseline ROC at almost every threshold. AUC score is 0.8510, a modest jump from the baseline AUC of 0.846, but the more important effect is that the overfit gap halved from 1.7% to 0.93%, meaning the tuned model's test performance is now far more trustworthy as an estimate of real-world behaviour. The early-FPR region rises more steeply, indicating the model is more confident when it does flag a defaulter, which is exactly the property you want for a credit scoring tool. Of 2,707 actual defaulters in the test set, the tuned model catches 1,974 (72.9%) and misses 733 (27.1%) defaulters.

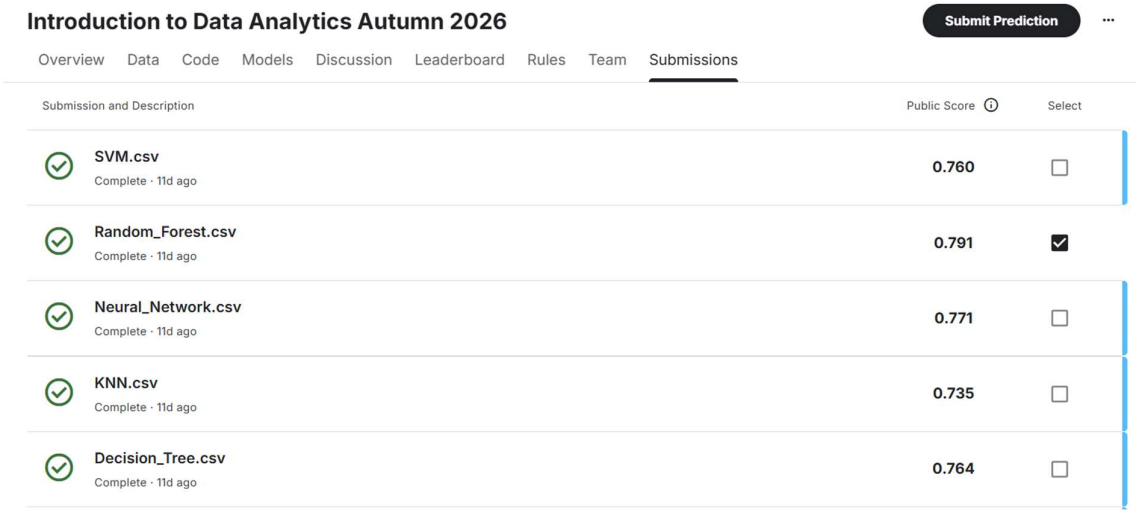








KAGGLE SUBMISSION



Model Evaluation and Results

All five models were evaluated on the identical held-out test set (n = 7,000, stratified) using six metrics: accuracy, precision, recall, F1-score (for the default class and macro-averaged), and ROC-AUC.

Model	*Kaggle score*
Decision Tree	*0.765*
KNN	*0.735*
Random Forest	*0.791*
SVM	*0.760*
Neural Network	*0.771*

Metric	*DT*	*KNN*	*RF*	*SVM*	*NN*
Accuracy	*0.8164*	*0.8013*	*0.8337*	*0.8016*	*0.8263*
Precision (Def.)	*0.7606*	*0.7765*	*0.7848*	*0.7365*	*0.8034*
Recall (Def.)	*0.7665*	*0.6827*	*0.7854*	*0.7580*	*0.7292*
F1 (Default)	*0.7636*	*0.7266*	*0.7851*	*0.7471*	*0.7645*

F1-macro	*0.8068*	*0.7852*	*0.8247*	*0.7919*	*0.8135*
ROC-AUC	*0.8365*	*0.8365*	*0.8544*	*0.8444*	*0.8510*
Total Errors	*1,285*	*1,391*	*1,164*	*1,389*	*1,216*

Model	*CV AUC Mean*	*CV AUC Std*	*CV F1m Mean*	*CV F1m Std*
Decision Tree	*0.8358*	*0.0052*	*0.8034*	*0.0065*
KNN	*0.8426*	*0.0039*	*0.7844*	*0.0020*
Random Forest	*0.8549*	*0.0034*	*0.8164*	*0.0012*
SVM	*0.8459*	*0.0055*	*0.7965*	*0.0034*
Neural Network	*0.8464*	*0.0052*	*0.7980*	*0.0059*

Table 2: Three-fold cross-validation results.

Looking at the error breakdown, Random Forest stands out for having an almost perfectly balanced error profile: 583 false positives and 581 false negatives. No other model comes close to this symmetry. The Neural Network produces the fewest false positives (483) but misses more actual defaults (733 false negatives), making it a conservative classifier. SVM goes the other direction, thus generating the most false positives (734) out of any model. KNN misses nearly a third of all defaults (859 false negatives), which rules it out for any application where catching defaulters matters.

An interesting pattern shows up in the overfit gaps. The Neural Network has the tightest gap (0.93%), the Decision Tree sits at 2.20%, and Random Forest is the widest at 4.20%. Yet the model with the widest gap is the best performer on unseen data. This happens because Random Forest's individual trees are deeper and more expressive, so each one fits its particular bootstrap sample quite tightly. But when 200 of these overfit-prone trees vote together, their individual mistakes cancel out. The cross-validation standard deviation confirms this: Random Forest's AUC varies by only 0.0034 across folds, the smallest spread of any model.

Best Classifier

Random Forest is the best model for this task. On raw performance, it leads on five of six test metrics and produces the fewest total errors (1,164 out of 7,000). The gap over the runner-up Neural Network is not just numerical but statistically confirmed (McNemar $p = 0.018$). In cross-validation, its AUC standard deviation of 0.0034 is the tightest of any model, meaning the results are reproducible rather than dependent on one lucky split. On the precision-recall scatter, it is the only model sitting in the zone where both precision and recall are near 0.785; every other model trades one off for the other. And it spreads its decision-making across more than 20 features, rather than depending heavily on one or two variables the way the single Decision Tree doe

Appendix

Decision Tree code

Decision Tree

```
# Baseline (unrestricted)
dt_base = DecisionTreeClassifier(random_state=42, class_weight='balanced')
dt_base.fit(X_train, y_train)
print(f'Baseline: depth={dt_base.get_depth()}, leaves={dt_base.get_n_leaves()}')
print(f'Train acc: {accuracy_score(y_train, dt_base.predict(X_train)):.4f}')
print(f'Test acc:  {accuracy_score(y_test, dt_base.predict(X_test)):.4f}')
```

```
# Hyperparameter tuning
dt_grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42, class_weight='balanced'),
    {'criterion':['gini','entropy'], 'max_depth':[5,7,9,11,13],
     'min_samples_leaf':[1,5,10,15,25], 'min_samples_split':[2,10,20]},
    cv=StratifiedKFold(3, shuffle=True, random_state=42),
    scoring='f1_macro', n_jobs=-1, verbose=1
)
dt_grid.fit(X_train, y_train)
print(f'\nBest params: {dt_grid.best_params_}')
print(f'Best CV F1-macro: {dt_grid.best_score_:.4f}')
```

KNN

K-Nearest Neighbours (KNN)

```
# Baseline
knn_base = KNeighborsClassifier(n_neighbors=5, n_jobs=-1)
knn_base.fit(X_train_s, y_train)
print(f'Baseline KNN (k=5): test acc = {accuracy_score(y_test, knn_base.predict(X_test_s)):.4f}')
```

```
# Tuning
knn_grid = GridSearchCV(
    KNeighborsClassifier(n_jobs=-1),
    {'n_neighbors':[5,11,21,31,51], 'metric':['euclidean','manhattan'], 'weights':['uniform','distance']},
    cv=StratifiedKFold(3, shuffle=True, random_state=42),
    scoring='f1_macro', n_jobs=-1, verbose=1
)
knn_grid.fit(X_train_s, y_train)
print(f'\nBest params: {knn_grid.best_params_}')
print(f'Best CV F1-macro: {knn_grid.best_score_:.4f}')
```

RF

Random Forest

```
# Baseline
rf_base = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced', n_jobs=-1)
rf_base.fit(X_train, y_train)
print(f'Baseline RF: train={accuracy_score(y_train, rf_base.predict(X_train)):.4f}, test={accuracy_score(y_test, rf_base.predict(X_test)):.4f}')
```

```
# Tuning
rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42, class_weight='balanced', n_jobs=-1),
    {'n_estimators':[100,200], 'max_depth':[9,12,15,None], 'max_features':['sqrt',0.5], 'min_samples_leaf':[1,5,10]},
    cv=StratifiedKFold(3, shuffle=True, random_state=42),
    scoring='f1_macro', n_jobs=-1, verbose=1
)
rf_grid.fit(X_train, y_train)
print(f'\nBest params: {rf_grid.best_params_}')
print(f'Best CV F1-macro: {rf_grid.best_score_:.4f}')
```

SVM

Support Vector Machine (SVM)

```
# Baseline
svm_base = SVC(kernel='rbf', C=1.0, gamma='scale', class_weight='balanced', random_state=42)
svm_base.fit(X_train_s, y_train)
print(f'Baseline SVM: test acc = {accuracy_score(y_test, svm_base.predict(X_test_s)):.4f}')
```

```
# Tuning on 8k subsample (SVM is O(n²))
X_sub, y_sub = resample(X_train_s, y_train, n_samples=8000, stratify=y_train, random_state=42)

svm_grid = GridSearchCV(
    SVC(class_weight='balanced', random_state=42),
    {'C':[0.1,1.0,2.0,5.0], 'kernel':['rbf','linear'], 'gamma':['scale']},
    cv=StratifiedKFold(3, shuffle=True, random_state=42),
    scoring='f1_macro', n_jobs=-1, verbose=1
)
svm_grid.fit(X_sub, y_sub)
print(f'\nBest params: {svm_grid.best_params_}')
print(f'Best CV F1-macro: {svm_grid.best_score_:.4f}')
```

Neural Network

Neural Network

```
# Baseline
nn_base = MLPClassifier(hidden_layer_sizes=(64,32), max_iter=200, random_state=42)
nn_base.fit(X_train_s, y_train)
print(f'Baseline NN: train={accuracy_score(y_train, nn_base.predict(X_train_s)):.4f}, test={accuracy_score(y_test, nn_base.predict(X_test_s)):.4f}, iters={nn_base.n_iter_}')
```

Python

```
# Tuning
nn_grid = GridSearchCV(
    MLPClassifier(solver='adam', max_iter=300, random_state=42, early_stopping=True, validation_fraction=0.1),
    {'hidden_layer_sizes':[(64,32),(128,64),(64,)], 'activation':['relu','tanh'], 'alpha':[0.0001,0.001,0.01]},
    cv=StratifiedKFold(3, shuffle=True, random_state=42),
    scoring='f1_macro', n_jobs=-1, verbose=1
)
nn_grid.fit(X_train_s, y_train)
print(f'\nBest params: {nn_grid.best_params_}')
print(f'Best CV F1-macro: {nn_grid.best_score_:.4f}')
```

Python